

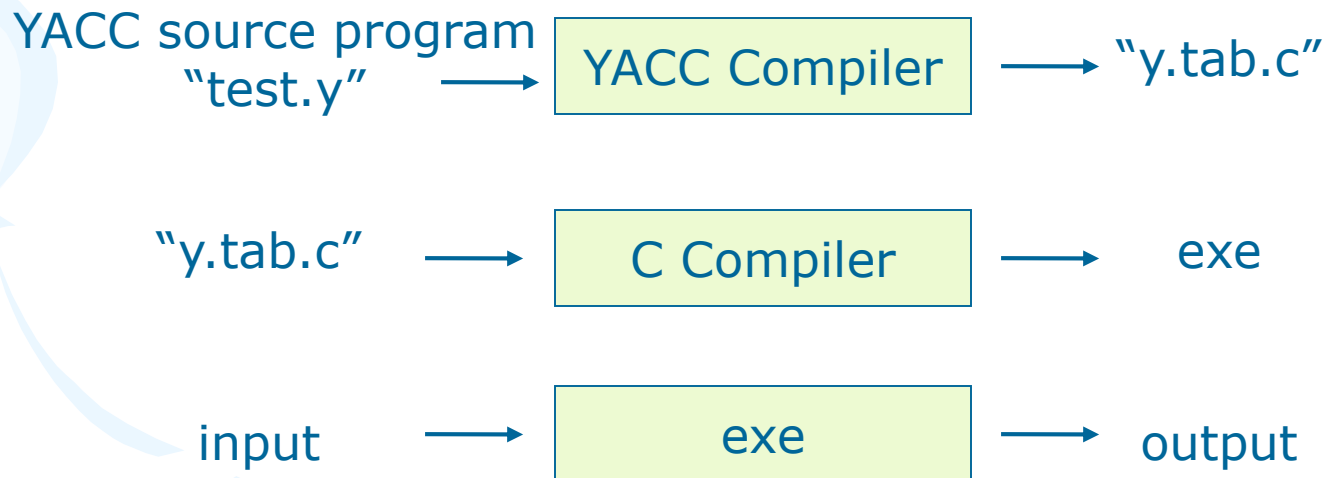
The background features several large, stylized, overlapping swirls in light green, light blue, and light purple. Scattered throughout are numerous small, yellow, four-pointed starburst shapes, some of which are larger and more prominent than others. The overall aesthetic is clean and modern.

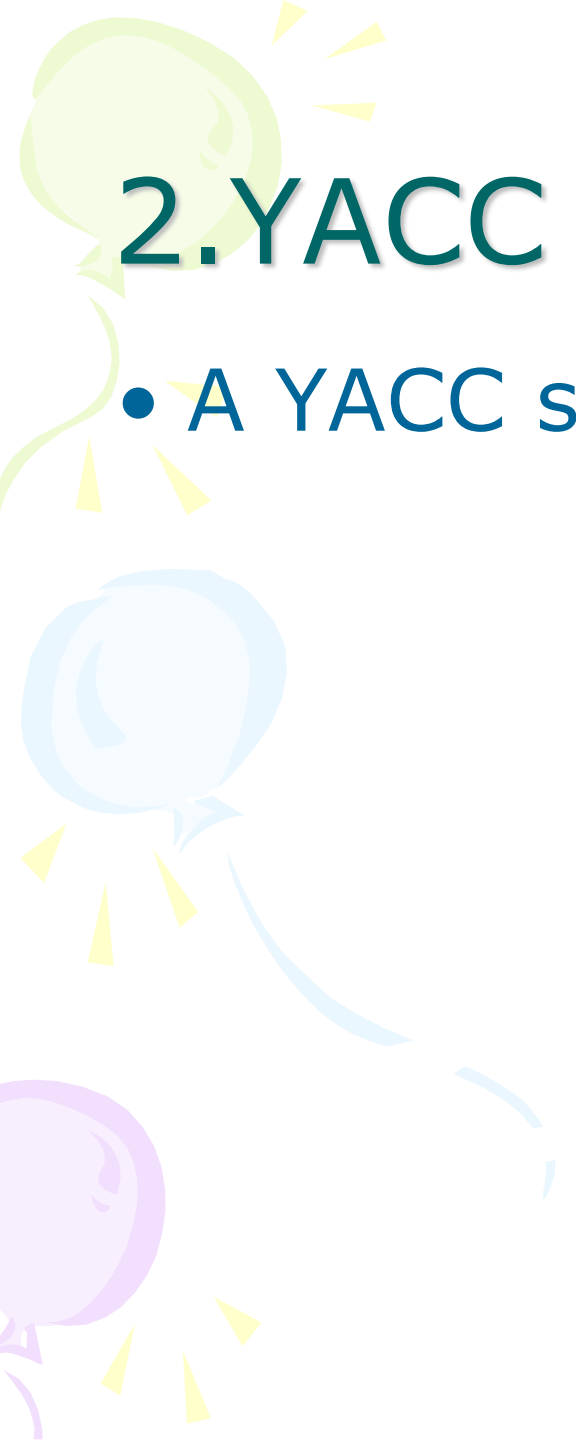
YACC

Yet Another Compiler-Compiler

1. How It Works

- A parser generator





2. YACC Structure

- A YACC source program has 3 parts:

declarations

%%

translation rules

%%

support C code

```
%{
#include <ctype.h>
#include <stdio.h>
#define YYSTYPE double /* Used for Yacc stack */
%}
```

Declaration

```
%token NUMBER
%left '+' '-'
%left '*' '/'
%left UMINUS
```

lowest first logic

%%	production rules	semantic actions
lines	: lines expr '\n'	{ printf("%g\n", \$2); }
	lines '\n'	
	/* this is a production of the empty word */	
	;	

Translation Rules

expr	: expr '+' expr	{ \$\$ = \$1 + \$3; }
	expr '-' expr	{ \$\$ = \$1 - \$3; }
	expr '*' expr	{ \$\$ = \$1 * \$3; }
	expr '/' expr	{ \$\$ = \$1 / \$3; }
	'(' expr ')'	{ \$\$ = \$2; }
	'-' expr %prec UMINUS	{ \$\$ = - \$2; }
	NUMBER	

left hand side ; right hand side

%%

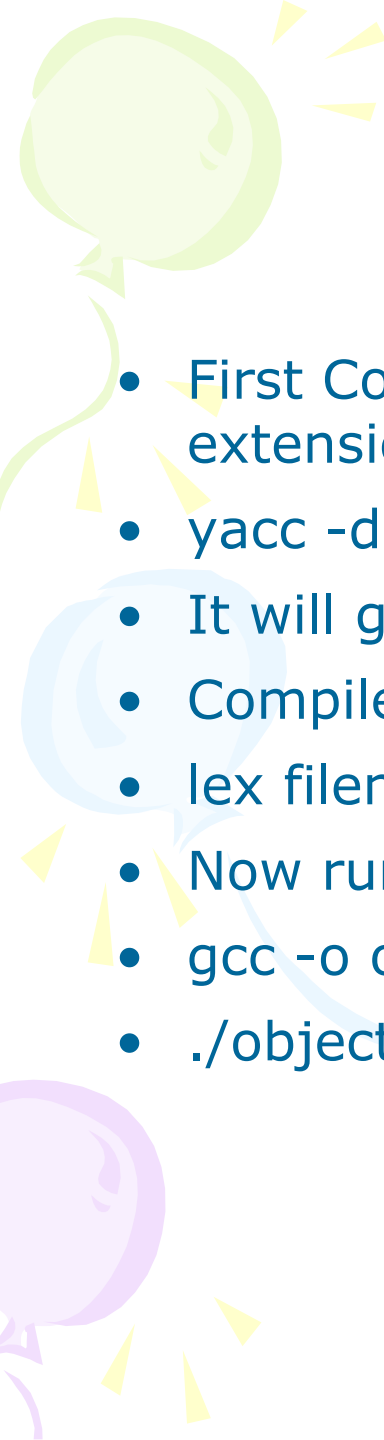
User-defined Supporting C code

```
yylex() {
    int c;
    while ((c = getchar()) == '');
    if ((c == '.') || (isdigit(c))) {
        ungetc(c, stdin);
        scanf("%lf", &yylval);
        return NUMBER;
    }
    return c;
}
```



3. LEX & YACC

- LEX: a tool for automatically generating a lexer or scanner given a lex specification (.l file); A lexer or scanner is used to perform lexical analysis, or the breaking up of an input stream into meaningful units, or tokens.
- YACC: a tool for automatically generating a parser given a grammar written in a yacc specification (.y file); A grammar specifies a set of production rules, which define a language, and corresponding actions to perform the semantics.



How to run yacc

- First Compile yacc program(open the file with .yacc extension)
- `yacc -d filename.yacc`
- It will generate two files- `y.tab.c` and `y.tab.h`
- Compile lex file.
- `lex filename.lex`
- Now run the below commands:
- `gcc -o objectFileName y.tab.c lex.yy.c -lfl`
- `./objectFileName < inputFile`

Example of yacc programs

Example 1. Program to recognize nested IF control statements and display the levels of nesting.

Yacc Part

```
%token IF RELOP S NUMBER ID
%{
    int count=0;
}%
%%
stmt : if_stmt { printf("No of nested if statements=%d\n",count); exit(0);}
      ;
if_stmt : IF '(' cond ')' if_stmt {count++;}
        | S;
        ;
cond : x RELOP x
      ;
x : ID
  | NUMBER
  ;
%%
int yyerror(char *msg)
{
    printf("Invalid Expression\n");
    exit(0);
}
main ()
{
    printf("Enter the statement");
    yyparse();
}
```



Lex Part

```
%{  
    #include "y.tab.h"  
}%  
%%  
"if" { return IF; }  
[sS][0-9]* {return S;}  
"<" ">" "=" "!=" "<=" ">=" { return RELOP; }  
[0-9]+ { return NUMBER; }  
[a-zA-Z][a-zA-Z0-9_]* { return ID; }  
\n { ; }  
. { return yytext[0]; }  
%%
```


Example 2. Program to recognize strings 'aaab', 'abbb', 'ab' and 'a' using grammar ($a^n b^n, n \geq 0$)

Yacc Part

```
%token A B NL
%%
stmt : s NL { printf("Valid String\n"); exit(0); }
      ;
s : A s B
  |
  ;
%%
int yyerror(char *msg)
{
    printf("Invalid String\n");
    exit(0);
}
main ()
{
    printf("Enter the String\n");
    yyparse();
}
```

Lex Part

```
%{
    #include "y.tab.h"
}%
%%
[aA] { return A; }
[bB] { return B; }
\n { return NL; }
. { return yytext[0]; }
%%
```